

SYSTEM DESIGN

Chapter 61: Design a Stock Trading Platform

Personal Notes

Payment systems (Ch 60) taught you the vocabulary of correctness under distributed pressure — idempotency, ledgers, sagas. Trading platforms take those requirements and add a nightmare: microsecond latency. When you tap "buy Nifty" in the Trackk app, an ORDER MATCHING ENGINE has to place your order into a book alongside millions of others, potentially match it instantly against a resting sell order, update positions across the entire user base, publish market data feeds to thousands of subscribers, and enforce risk limits — all within a few milliseconds end-to-end (microseconds for institutional venues).

This chapter designs a trading platform end-to-end. It covers the ORDER BOOK data structure (the heart of every exchange), the MATCHING ENGINE algorithm (price-time priority), MARKET DATA FEEDS (how prices propagate to all participants), POSITION MANAGEMENT (each trade updates every affected trader's holdings), and SETTLEMENT (the T+1/T+2 back-office reality). Along the way we'll cover the unique performance requirements — why exchanges use single-threaded matching engines, sub-microsecond networking, and specialized hardware. Beyond trading, these patterns power any real-time auction system, order matching in Uber, ad-serving auctions.

1. The Problem

Design a stock trading platform. Users place BUY and SELL orders for financial instruments (stocks, futures, options). The system MATCHES compatible orders (buyer meets seller), executes TRADES, updates POSITIONS for both counterparties, publishes MARKET DATA (price/volume) to all subscribers, and eventually SETTLES the trade with the depository. Handle thousands of orders per second per instrument, with millisecond latency and iron-clad consistency.

```
Alice: BUY 100 shares of RELIANCE @ 2450 (limit order)
Bob:   SELL 100 shares of RELIANCE @ 2440 (limit order)
      |
      ▼
Matching engine:
  Alice's BID 2450 >= Bob's ASK 2440 → MATCH
  Trade executes at BOB's price (better for Alice)
  Trade printed: 100 shares @ 2440
      |
      ├──> Update Alice's position: +100 RELIANCE, -244,000 INR
      ├──> Update Bob's position:  -100 RELIANCE, +244,000 INR
      ├──> Publish MARKET DATA: RELIANCE last trade = 2440
      └──> Send trade confirmations to both traders
```

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Place, modify, and cancel orders (buy/sell)
- Multiple order types: MARKET (execute at best price), LIMIT (only at my price or better), STOP LOSS
- Match compatible orders using price-time priority
- Publish real-time market data (last price, best bid/ask, top-of-book depth)
- Maintain user portfolios (holdings + P&L)
- Handle settlement (T+1 for stocks in India, T+2 elsewhere)
- Pre-trade risk checks (has user enough funds? approved to trade this?)

2.2 Non-Functional Requirements

- Retail broker (Trackk, Zerodha, Groww): ~1000 orders/sec, ~10 ms end-to-end order-to-ack latency
- Retail exchange (NSE, BSE): ~100K orders/sec, sub-millisecond matching latency
- HFT venue (NYSE, LSE): ~1M orders/sec, sub-microsecond matching
- CORRECTNESS — every trade recorded exactly once; positions must always balance
- FAIRNESS — orders processed in time-order; no jumping the queue
- AUDITABILITY — every order + trade retained for regulatory review (7+ years)

2.3 Constraints to Confirm

- Are we the BROKER (routes orders to an exchange) or the EXCHANGE (matches orders)?
- Instruments: stocks only, or also derivatives (F&O)?
- Market hours only, or 24/7 (crypto)?
- Cash market only, or margin/leverage?

The critical clarification — broker vs exchange: A BROKER (Trackk, Zerodha, Robinhood) receives customer orders and routes them to an exchange. It doesn't match orders itself. An EXCHANGE (NSE, NASDAQ, BSE) actually matches buyers and sellers. Very different systems: brokers focus on onboarding + risk + UX; exchanges focus on the matching engine. Confirm which one you're designing before proceeding.

3. Step 2 — Capacity Estimation

Order Throughput (Broker vs Exchange)

Retail broker (Trackk-scale):
 $1\text{M active users} \times 5 \text{ orders/day} = 5\text{M orders/day}$
 $\div 6 \text{ hours market open} = \sim 230 \text{ orders/sec average}$

× 10 peak factor (opening bell) ≈ 2,300 orders/sec peak

Retail exchange (NSE):

Aggregates all brokers → ~100K orders/sec peak

Peak observed during high-vol events: 250K/sec

Every order generates market data update – read amplification is huge.

Market Data Fanout

Each order/trade updates the order book → produces market data event
Subscribers to that instrument = potentially every user watching it

For a popular stock like RELIANCE:

100K concurrent viewers subscribed to level-1 data (best bid/ask)

10K subscribers to level-2 (order book depth)

Trade rate on RELIANCE: 100 trades/sec

Fanout: 100 events/sec × 100K subscribers = 10M messages/sec

Multicast (UDP-based feeds) is the standard for large exchanges.

Retail brokers use WebSocket per user (100K persistent connections).

Storage

Orders per day: 10M average across brokerage industry

Retained forever for regulatory compliance → ~4B orders/year

Per order: ~1 KB → ~4 TB/year

Trades: ~100K/day → 40M/year

Per trade: ~500 B → ~20 GB/year

Market data feed: ~10M events/sec × 100 bytes = ~1 GB/sec of tick data

Compressed + retained selectively → still petabytes over years.

Position state (small): ~500 B per user × 1M users = 500 MB total.

4. Step 3 — API Design

Order APIs (mostly synchronous, low-latency)

POST /v1/orders

Headers: Idempotency-Key: uuid-1234

```
{
  "user_id": "u_42",
  "instrument": "RELIANCE",
  "side": "BUY",
```

```

    "order_type": "LIMIT",      # or MARKET, STOP_LOSS
    "quantity": 100,
    "price": 2450.50,          # limit price (null for MARKET)
    "time_in_force": "DAY"     # or IOC (immediate-or-cancel), FOK (fill-or-kill)
  }
→ 201 { "order_id": "ord_789", "status": "OPEN", "filled_qty": 0 }

PUT /v1/orders/ord_789/modify
{ "new_price": 2455 }
→ 200 { "order_id": "ord_789", "new_price": 2455 }

DELETE /v1/orders/ord_789      # cancel
→ 200 { "order_id": "ord_789", "status": "CANCELLED" }

GET /v1/positions?user_id=u_42
→ 200 { "positions": [
  { "instrument": "RELIANCE", "qty": 100, "avg_price": 2440.00,
    "current_price": 2445.00, "pnl": 500 }
  ]}

# Market data (streaming via WebSocket)
WS /market-data
→ SUBSCRIBE {"instruments": ["RELIANCE", "TCS"], "level": 1}
← EVENTS {"instrument": "RELIANCE", "bid": 2439.5, "ask": 2440.5,
"last": 2440}

```

The Idempotency-Key returns: Same principle as Ch 60 (payments). If a client's order request times out, they must be able to retry safely. The idempotency key ensures the retry gets the ORIGINAL order back, not a duplicate. Without this, a network flap during a buy order could result in two positions instead of one — a serious problem when markets are moving fast.

5. Step 4 — Data Model

```

Table: orders
  order_id (PK) | user_id | instrument | side | type | qty | price
                | status | created_at | idempotency_key
  Sharded by instrument (all orders for RELIANCE co-located).
  Index on user_id for portfolio queries.

Table: trades
  trade_id (PK) | buy_order_id | sell_order_id | instrument
                | qty | price | executed_at
  APPEND-ONLY. Immutable. Regulatory archive.

Table: order_book_state (in-memory only)    ← Per instrument
  Two sorted structures:
    bids: sorted by price DESC, then time ASC

```

asks: sorted by price ASC, then time ASC
NOT persisted – rebuilt from open orders on startup.

Table: positions ← Per user, per instrument
user_id | instrument | qty | avg_cost | realized_pnl
Updated on every trade (double-entry ledger style, see Ch 60).

Table: user_balance (from Ch 60 ledger concepts)
user_id | currency | balance | reserved (for pending orders)

Table: risk_limits
user_id | max_notional_per_order | max_open_orders |
trading_permissions

Time-series store (Kafka/InfluxDB) for market data ticks:
(instrument, timestamp) → { bid, ask, last, volume, ... }

6. The Core Concept — The Order Book

The ORDER BOOK is the fundamental data structure of every exchange. It holds all UNMATCHED (open) orders, split into two sides: BIDS (buyers) and ASKS (sellers). Each side is sorted by price — bids by highest first, asks by lowest first. When a new order arrives, the matching engine checks if it can cross with the opposite side.

Order book for RELIANCE (snapshot):

ASKS (sellers)		BIDS (buyers)		
Price	Qty	Price	Qty	
2445.00	500	2440.00	300	← BEST BID
(highest)				
2447.50	200	2438.50	1000	
2450.00	1000	2436.00	400	
2455.00	100	2430.00	2000	
...		...		

BEST ASK (lowest) = 2445.00 (500 shares available)

BEST BID (highest) = 2440.00 (300 shares wanted)

SPREAD = 2445.00 - 2440.00 = 5.00 rupees

No trade until a new BID arrives at ≥ 2445 OR a new ASK at ≤ 2440 .

Order Book Rules

- PRICE PRIORITY — better prices execute first (highest bid, lowest ask)
- TIME PRIORITY — at the SAME price, earlier orders execute first (FIFO within price level)
- Both together = PRICE-TIME PRIORITY, the standard for most exchanges
- Some markets use PRO-RATA (matching split by size, not time) — rarer

7. Order Book Data Structures

How to represent the order book in memory for maximum speed? Two-level structure is the industry standard.

Two-Level Approach

```
class OrderBook {
    // Outer: sorted by price. For bids DESC, asks ASC.
    TreeMap<Long, PriceLevel> bids = new
TreeMap<>(Collections.reverseOrder());
    TreeMap<Long, PriceLevel> asks = new TreeMap<>();
    // Price stored as long (fixed-point: paise not rupees) – avoid float!
}

class PriceLevel {
    long price;
    long totalQty;                // sum of all order qtys at this level
    Deque<Order> orders;          // FIFO queue at this price
}

class Order {
    long orderId;
    long userId;
    long qty;
    long price;
    long timestamp;
    Side side;
}
```

Why This Structure

- Outer TreeMap: sorted price levels — $O(\log P)$ to find best price, where P = distinct prices
- Inner Deque: FIFO at each price → time priority in $O(1)$
- addOrder: $O(\log P)$ to find price + $O(1)$ to append
- matchOrder: $O(1)$ to peek best price + $O(1)$ to remove head
- cancelOrder: $O(1)$ with a hashmap of order_id → position

Never use floating-point for prices: $0.1 + 0.2 = 0.30000000000000004$ in IEEE 754. In finance, that half-cent difference can cause auditors nightmares. Store prices as INTEGERS in the smallest unit (paise for INR, cents for USD). Convert to human display only at the edges. Every serious exchange uses fixed-point arithmetic — do not deviate.

8. Deep Dive: The Matching Engine Algorithm

When a new order arrives, the matching engine tries to CROSS it against the opposite side of the book. Match at the resting order's price (better for the aggressor). Continue crossing until the incoming order is fully filled or the book has no more matching prices.

Algorithm

```
public List<Trade> processOrder(Order incoming) {
    List<Trade> trades = new ArrayList<>();
    long remaining = incoming.qty;

    // Determine the book side we're matching AGAINST
    TreeMap<Long, PriceLevel> opposite =
        incoming.side == BUY ? asks : bids;

    // Match: for each price level from best downwards,
    // consume orders until we fill or price becomes unfavorable
    while (remaining > 0 && !opposite.isEmpty()) {
        Long bestPrice = opposite.firstKey();

        // Check if the incoming order accepts this price
        if (incoming.type == LIMIT) {
            if (incoming.side == BUY && bestPrice > incoming.price) break;
            if (incoming.side == SELL && bestPrice < incoming.price)
                break;
        }
        // MARKET orders accept any price (up to book depth)

        PriceLevel level = opposite.get(bestPrice);
        while (remaining > 0 && !level.orders.isEmpty()) {
            Order resting = level.orders.peekFirst();
            long fillQty = Math.min(remaining, resting.qty);

            trades.add(new Trade(incoming.orderId, resting.orderId,
                                bestPrice, fillQty));
            resting.qty -= fillQty;
            remaining -= fillQty;
            level.totalQty -= fillQty;

            if (resting.qty == 0) {
                level.orders.pollFirst(); // remove filled order
            }
        }

        if (level.orders.isEmpty()) opposite.remove(bestPrice);
    }

    // Any remaining qty rests in the book (LIMIT only; MARKET expires)
```

```

    if (remaining > 0 && incoming.type == LIMIT) {
        addToBook(incoming, remaining);
    }
    incoming.qty = remaining;
    return trades;
}

```

Example — Alice's Order Meets the Book

Before: RELIANCE book
 ASKS: 2445 (500), 2447.50 (200), 2450 (1000)
 BIDS: 2440 (300), ...

Alice sends: BUY 700 @ LIMIT 2448

Matching engine:
 Best ask 2445 acceptable (≤ 2448) → match
 Consume 500 @ 2445 → trade 500 @ 2445 → remaining 200

Next best ask 2447.50 acceptable → match
 Consume 200 @ 2447.50 → trade 200 @ 2447.50 → remaining 0

Result:
 Alice bought 500 @ 2445 + 200 @ 2447.50 = 700 shares
 Average fill price: 2445.71
 Book after:
 ASKS: 2450 (1000) ← 2445 and 2447.50 levels wiped out
 BIDS: unchanged

Why matching engines are single-threaded: The matching engine must process orders in a strict TIME ORDER for fairness. Multi-threaded matching creates races — two threads could match orders in reverse order of arrival. Every serious exchange runs the matching engine on a SINGLE THREAD pinned to a CPU core, processing millions of orders/sec with careful data locality (LMAX Disruptor pattern). NASDAQ, NYSE, CME all follow this design.

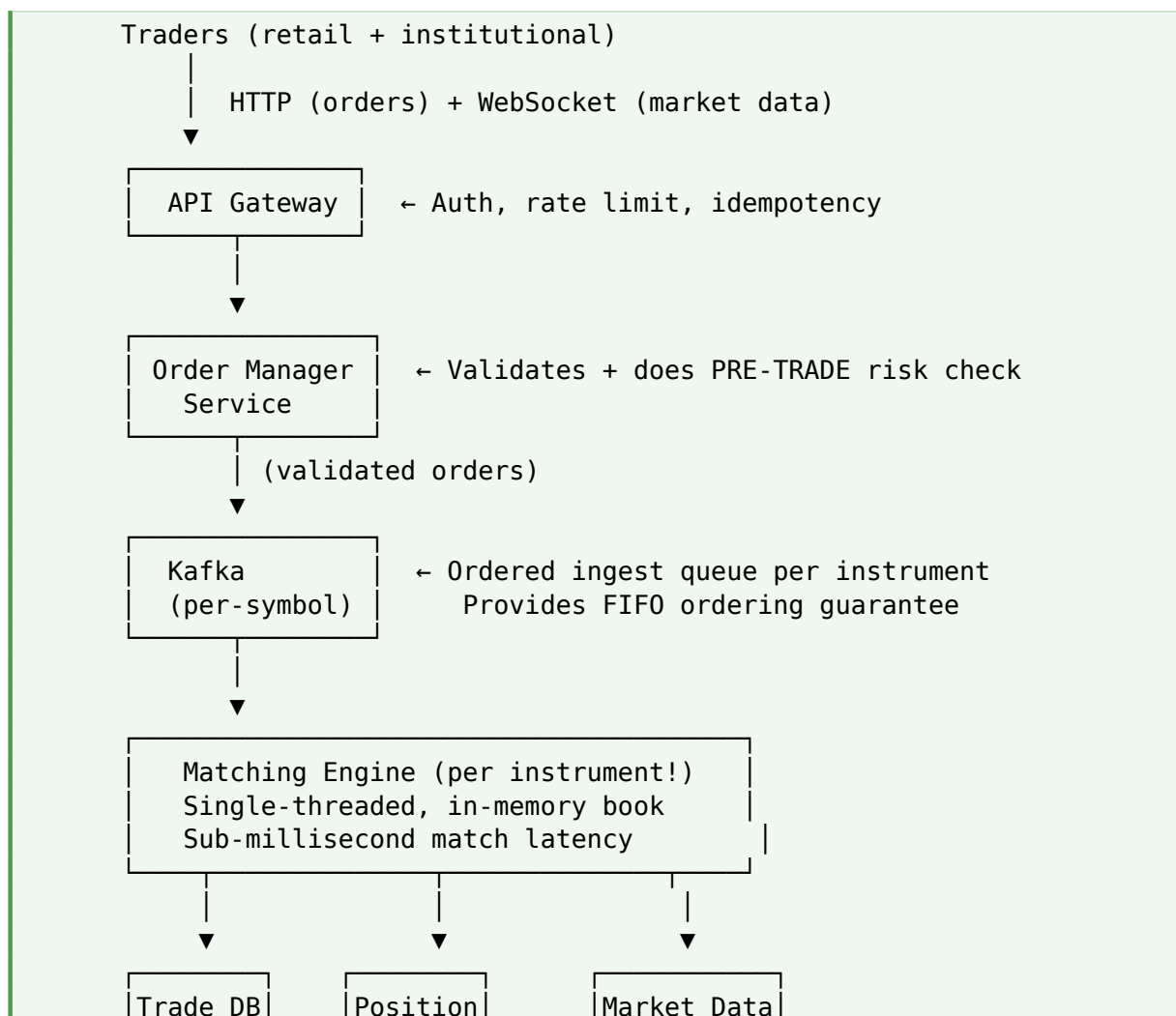
9. Deep Dive: Order Types

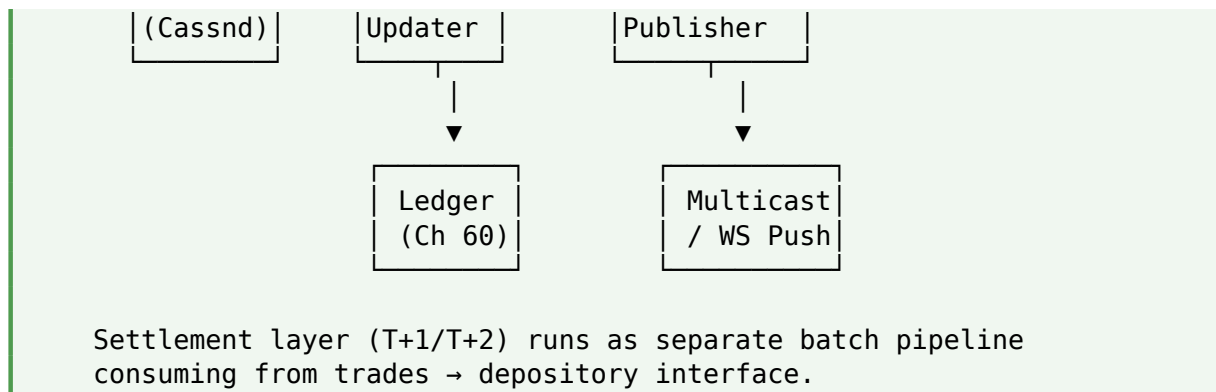
Beyond simple market and limit orders, exchanges support many order types. Each has slightly different matching semantics.

Type	Semantics	Used For
MARKET	Execute at whatever price is best in the book	Guaranteed fill, unknown price
LIMIT	Execute only at my price or better; otherwise rest in book	Known price, may not fill

Type	Semantics	Used For
STOP LOSS	Once trigger price is hit, convert to MARKET/LIMIT order	Protect against big losses
IOC	Immediate or cancel — fill what's possible, cancel rest	Reduce market impact
FOK	Fill or kill — must fill entirely or cancel	Institutional block trades
ICEBERG	Only show part of the size publicly; hide the rest	Hide large intentions from market
GTC	Good-till-cancel — stays in the book across days	Long-term price targets

10. Step 5 — High-Level Architecture





Service Roles

- Order Manager — pre-trade risk (fund check, permissions, limits) + validation
- Kafka — provides ORDERED ingest per instrument, isolates the matching engine from bursts
- Matching Engine — single-threaded per instrument, holds order book in memory
- Trade Publisher — writes trades to durable store, publishes to downstream consumers
- Position Updater — consumes trades, updates positions + ledger (Ch 60 pattern)
- Market Data Publisher — pushes book updates to subscribers

11. Deep Dive: Sharding by Instrument

One matching engine can't handle every stock — the single-threaded design caps at maybe 500K orders/sec on a fast machine. Solution: run ONE matching engine per instrument (or per group of instruments), each on its own dedicated hardware/process. Sharding by instrument is natural because ORDERS FOR DIFFERENT INSTRUMENTS DON'T INTERACT.

Sharding topology:

Matching Engine 1 (Server A):
RELIANCE, TCS, HDFC, INFY, ITC (5 high-volume stocks)
Handles ~300K orders/sec total

Matching Engine 2 (Server B):
SBIN, LTIM, KOTAKBANK, WIPRO, ONGC (5 more)

Matching Engine 3 (Server C):
...

For high-volume stocks (RELIANCE alone might be 100K/sec at peak), get its OWN dedicated engine.

Failover: passive standby with same instrument allocation, ready to take over on primary failure.

Sharding by user or trade would be WRONG: Every order for RELIANCE must go through THE SAME order book — because they need to be compared against each other for matching. Sharding by user (which we did for chat in Ch 53) makes no sense here. INSTRUMENT is the only correct shard key. This constraint drives the whole architecture.

12. Deep Dive: Market Data Distribution

Every match, every new resting order, every cancel — all produce a MARKET DATA EVENT. These events must reach every interested participant in real-time. At Trackk-scale that's 100K WebSocket subscribers. At NSE-scale that's millions.

Distribution Layers

Feed Level	Contents	Audience
Level 1	Best bid + best ask + last trade	Retail traders on brokerage apps
Level 2	Top 5-10 levels of the order book	Retail power users, brokers
Level 3	Full order book with individual orders	Institutional, HFT
Trade tape	Every executed trade with price + qty	Everyone; also used for TV tickers

Distribution Mechanisms

- Multicast UDP (exchange-grade) — one packet reaches every subscriber; sub-microsecond fanout
- Pub/Sub over TCP (broker-grade) — WebSocket or SSE per user; reliable but slower fanout
- Throttled snapshots — instead of every tick, send diff snapshots every 100ms for slow subscribers

Snapshotting matters for slow subscribers: A 3G mobile user can't consume 10K market data ticks per second. Broker apps typically throttle — send BEST BID/ASK at most every 100 ms, buffered on server-side. Institutional feeds send raw. Real-time doesn't mean "every tick" — it means "latest state, delivered quickly."

13. Deep Dive: Position Management

Every trade updates positions for BOTH counterparties. If Alice buys 100 shares from Bob, Alice's position goes +100 and Bob's goes -100. The position updater is downstream of the matching engine — it doesn't need to be in the critical latency path (positions are eventually consistent for display; they're only strictly enforced on next-order risk check).

Position Calculation

```
class Position {
    long userId;
    String instrument;
    long qty;                // net position (positive = long, negative =
short)
    long avgCost;            // weighted average price of accumulated qty
    long realizedPnl;        // gains/losses from closed trades
    // unrealizedPnl computed on-the-fly from current market price
}

void applyTrade(Position p, Trade t) {
    if (t.side == BUY) {
        // Weighted average
        long newQty = p.qty + t.qty;
        p.avgCost = (p.avgCost * p.qty + t.price * t.qty) / newQty;
        p.qty = newQty;
    } else {
        // Selling: realize P&L based on avg cost
        long pnl = (t.price - p.avgCost) * t.qty;
        p.realizedPnl += pnl;
        p.qty -= t.qty;
    }
}
```

Position Storage

- Hot state: Redis or in-memory service — every trade updates the current position immediately
- Durable state: SQL DB — updated every N seconds, or via async event log
- Trade history: append-only in Cassandra/Kafka for auditability + replay

Positions must be CONSISTENT with trades: Sum of ALL positions in the system must equal sum of ALL trades (net). This is an INVARIANT — if it doesn't hold, something is broken. Continuous reconciliation (similar to Ch 60) — compute positions from trade log periodically and compare with stored positions. Alert on drift.

14. Deep Dive: Pre-Trade Risk Checks

Before an order enters the matching engine, verify the trader has the funds/margin/permissions. This is the ORDER MANAGER's job. Risk checks must be FAST (adds latency to every order) and CONSERVATIVE (better to reject a valid order than accept an invalid one).

Standard Checks

- Funds — does user have enough balance to cover this buy? (For sells: do they own the shares?)
- Margin — for derivatives, check margin requirement vs available
- Position limits — max long/short position for this user for this instrument

- Notional limits — max order size in currency (prevents fat-finger errors)
- Permissions — is this user allowed to trade this instrument?
- Market hours — is the market open for this instrument?
- Kill switch — has this user been flagged?

Latency Trick — Optimistic Reserve

Rather than atomically checking-and-updating balance on every order (a slow point of contention), OPTIMISTICALLY reserve funds when the order enters, then release/settle after the trade completes or the order is cancelled.

```
# Order comes in for BUY 100 @ 2450 (notional 245,000)
1. reserved += 245,000      # atomic increment on user's reserved balance
2. IF available_balance - reserved < 0 → REJECT + refund reservation
3. Submit order to matching engine
4. On fill: convert reservation to actual debit (per Ch 60 ledger)
5. On cancel: release reservation
```

15. Deep Dive: Settlement & Clearing

A trade EXECUTES instantly, but SETTLEMENT (actual movement of shares between depository accounts + funds) happens later. In India, most stock trades settle T+1 (next business day). Options settle same day. This means the exchange bears counterparty risk during that window.

The Clearing Corporation

- Sits between buyer and seller — becomes the counterparty to both
- Aggregates all trades for each broker at end of day → NET obligation per broker
- Broker posts collateral (margin) proportional to their net position
- On settlement day, moves cash + shares via depository

Trading day:

Alice buys 100 RELIANCE from Bob @ 2440

End of day (T+0):

Clearing corp calculates NET obligations

Alice's broker: owes 244,000 cash, gets 100 RELIANCE

Bob's broker: owes 100 RELIANCE, gets 244,000 cash

Margins are locked

T+1:

Cash moves via RBI's RTGS/NEFT to broker accounts

Shares moved via NSDL/CDSL from Bob's demat to Alice's demat

Trade is SETTLED. Both parties' positions final.

The broker's role in settlement: For retail brokers (Trackk), your position is with YOUR BROKER, not directly with the exchange. The broker posts collateral to the clearing corp on your behalf. If you

don't pay for shares by T+1, your broker sells them to cover (auto-square-off) — the exchange doesn't know or care about individual retail settlement, just the broker's net obligation.

16. Additional Features

16.1 Order Modifications

Modifying an order (price or qty) has TIME PRIORITY implications. Increasing qty or worsening price loses time priority (treated as a new order). Decreasing qty or improving price keeps priority. Most exchanges implement modify as "cancel + new" but with priority preservation rules.

16.2 Auction (Opening Session)

First 15 minutes of many exchange days is an AUCTION — orders accumulate without matching, then all match at once at the EQUILIBRIUM price. Prevents wild opening swings. Different matching algorithm (batch auction) than the continuous session that follows.

16.3 Circuit Breakers

If price moves too fast (e.g., 10% in 5 minutes), trading is HALTED. This is a regulatory circuit breaker. Design implication: matching engine must accept HALT commands and refuse to match while halted. Orders can still be entered/cancelled during halt.

16.4 Algorithmic Trading Interfaces

Institutional users don't use a web UI — they connect via FIX (Financial Information eXchange) protocol, sending millions of orders programmatically. Provide a FIX gateway alongside the REST API. FIX is verbose text over TCP; institutional-grade parsers handle sub-microsecond decoding.

16.5 Market Making + Liquidity Programs

Exchanges pay MARKET MAKERS to continuously quote both bid and ask. This narrows spreads and provides liquidity. Design: separate class of order ("MM quote") with lower fees and higher time-priority guarantees.

17. Failure Modes

Failure	Mitigation
Matching engine crash	Kafka retains all pending orders; standby engine takes over from last committed order
Position updater lag	Trades queue up; positions stale for display but new orders use last-known state

Failure	Mitigation
Market data outage	Trading continues (matching still works); traders see stale prices — halt is often triggered
Kafka down	Cannot accept new orders; existing engine continues on in-memory state. Halt new orders.
Erroneous fat-finger order	Pre-trade risk should catch (notional cap). Post-trade: BUST authority to cancel trades
Cross-shard settlement issue	Reconciliation the next day catches. Manual intervention if serious.
Regulatory halt	Exchange broadcasts halt; matching engine stops matching for that instrument
Broker default	Clearing corp uses posted collateral to make counterparties whole. Costly for the clearing corp.

18. Latency Requirements & Performance Tricks

Exchange-grade matching engines are among the highest-performance software on the planet. Achieving sub-microsecond latency requires special engineering.

Techniques Used by Top-Tier Exchanges

- Single-threaded matching engine pinned to a specific CPU core
- In-memory order book with fixed-size preallocated data structures (no GC pauses)
- LMAX Disruptor pattern for wait-free inter-thread communication
- Kernel bypass networking (Solarflare/Mellanox) — user-space TCP/UDP
- Colocation — matching engine in same rack as high-frequency traders' machines
- FPGA-accelerated market data parsing and order entry
- Business logic in C++ or Java Zing/Azul (concurrent GC with sub-ms pauses)

You don't need this at Trackk-scale: For a retail broker like Trackk (10 ms end-to-end latency), standard Java + Kafka + PostgreSQL + Redis is fine. The exotic tricks above are for institutional exchanges where microseconds matter for HFT profitability. But KNOW they exist — a good interviewer might ask "how would you go from millisecond to microsecond?" and the answer is these techniques.

19. Trade-offs Summary

Decision	Trade-off
Broker vs exchange	Broker: onboarding + UX + risk; exchange: matching + market data. Very different focus.
Price-time vs pro-rata matching	Price-time = fair + simple; pro-rata = better for large orders
Single-threaded vs multi-threaded	Single = correctness + simplicity; multi = throughput but complex fairness
In-memory book vs persistent	In-memory = fast; persistent = recovery from crash
Multicast vs pub-sub for feeds	Multicast = fanout, no reliability; pub-sub = reliable but slower
Optimistic reservation vs pessimistic	Optimistic = low latency; pessimistic = simpler but slower
Same-thread trade publish vs async	Same-thread = trade written before ack; async = faster but complex recovery
Colocation vs geographic distribution	Colo = low latency for HFT; distributed = fair for retail

20. Common Mistakes

Mistake 1: Not distinguishing broker vs exchange

These are fundamentally different systems. If you build a broker, you don't need a matching engine at all. If you build an exchange, matching IS your product. Clarify upfront.

Mistake 2: Multi-threaded matching engine

Wanting parallelism for throughput destroys FAIRNESS — orders may match in the wrong order relative to arrival time. Single-threaded per instrument with sharding by instrument is the standard answer.

Mistake 3: Using floating-point for prices

IEEE 754 rounding errors compound into audit nightmares. Use integer paise/cents throughout. Convert to human display only at rendering time.

Mistake 4: Sharding by user

You can't shard by user because orders from different users must be COMPARED (matched). Instrument is the only correct shard key. Bring this up unprompted.

Mistake 5: Skipping pre-trade risk

A trader without funds sending massive orders can cause chaos. Every order goes through pre-trade risk BEFORE the matching engine, even if it adds a few ms of latency. Adding pre-trade risk POST-execution is worse than not having any.

Mistake 6: No plan for market data throttling

At full-tilt tick rates, retail users' phones can't keep up. Server-side throttling with snapshots (every 100 ms) is essential. Streaming raw is only for institutional consumers.

Mistake 7: Not addressing regulatory obligations

Trading platforms are HEAVILY regulated. Every order + trade retained 7+ years. Audit trails mandatory. KYC/AML on onboarding. Circuit breakers on volatile moves. Mention these — regulators are part of your architectural constraints.

21. Best Practices

- Shard the matching engine BY INSTRUMENT — never by user or by trade
- Single-threaded matching engine per instrument — correctness + fairness > raw throughput
- Store prices as INTEGERS in smallest currency unit (paise, cents) — never floats
- Kafka in front of matching engine for ORDERED ingest + replay after crash
- In-memory order book with two-level structure (TreeMap of PriceLevels)
- Pre-trade risk check BEFORE the matching engine, with optimistic fund reservation
- Idempotency-Key on every order submission (per Ch 60)
- Publish market data via multicast (exchange) or WebSocket (broker)
- Throttle market data to slow subscribers — every-100ms snapshot vs every-tick raw
- Retain every order + trade FOREVER for regulatory audit

22. Quick Reference

Concept	Meaning
Order book	In-memory data structure holding open orders — bids + asks
Bid / Ask	Buyer's price / Seller's price
Best bid / ask	Highest bid + lowest ask; midpoint is the 'market price'
Spread	Ask minus bid — cost of round-trip trade
Price-time priority	Better price first; at same price, earlier order first
Matching engine	Single-threaded per instrument, matches incoming orders

Concept	Meaning
LIMIT order	Execute at my price or better; else rest in book
MARKET order	Execute at whatever price is best — accept the spread
Stop loss	Convert to market/limit once trigger price hit
Level 1/2/3 market data	Best bid+ask / top depth / full book
Optimistic reservation	Lock funds when order enters; release on cancel/settle
Settlement (T+1/T+2)	Actual movement of shares + cash, days after trade
Clearing corp	Central counterparty; manages settlement risk
FIX protocol	Institutional order-entry protocol over TCP

23. Related Interview Problems

Trading platform patterns apply to any real-time matching / auction system:

1. Design a Crypto Exchange (Binance, Coinbase) — same matching engine, 24/7, on-chain settlement
2. Design an Options Pricing Platform — same book but with derivatives math
3. Design a Prediction Market — order book on future events
4. Design an Ad Auction System (Google Ads) — real-time auction, sub-100ms latency
5. Design a Sports Betting Exchange (Betfair) — order book on match outcomes
6. Design a Ride-Share Matching (Ch 54 Uber) — many-to-many matching with location
7. Design a Real Estate Auction Platform — batch auctions with reserve prices
8. Design a Marketplace with Bidding (eBay) — asynchronous auction with deadline

24. Final Takeaway

Trading platforms sit at the intersection of every hard problem in distributed systems: microsecond latency, ironclad correctness, regulatory compliance, real-time data distribution to millions, and financial invariants that must always hold. The ORDER BOOK is the beating heart — a beautifully simple data structure (two sorted maps) that captures the essence of markets. The MATCHING ENGINE is the algorithm that makes markets function — deceptively straightforward code that runs on some of the fastest hardware ever built.

For interviews (especially at Trackk, Zerodha, Groww, or any trading-adjacent firm), memorize: (1) order book as TreeMap of price levels, each with a FIFO of orders, (2) matching algorithm with price-time priority, (3) sharding by instrument (never by user), (4) single-threaded matching for fairness +

correctness, (5) Kafka for ordered ingest + replay, (6) pre-trade risk with optimistic reservation, (7) tiered market data feeds. Then be ready for the deep-dives: "How do you achieve microsecond latency? What happens on crash? How do you handle a flash crash?"

Key Takeaway: Trading platform = ORDER BOOK + MATCHING ENGINE + MARKET DATA + POSITION MGMT + SETTLEMENT. Order book = TreeMap of price levels, each with FIFO of orders — bids DESC, asks ASC. Matching by PRICE-TIME PRIORITY: better price first, earlier order at same price. SINGLE-THREADED matching engine PER INSTRUMENT — never multi-threaded, never sharded by user. Prices are INTEGERS (paise, not floats). KAFKA provides ordered ingest + crash recovery. PRE-TRADE RISK check with optimistic reservation before engine. Market data via MULTICAST (exchange) or WEBSOCKET (broker), throttled for slow subs. SETTLEMENT is T+1/T+2, handled by clearing corp — separate from matching. Broker ≠ Exchange (clarify upfront!).

25. What's Next?

Trading platform done. Remaining designs and deep dives:

- Design a Notification System — cross-channel delivery, priorities, batching
- Design a Metrics/Monitoring System (Datadog-style) — time-series data at scale
- Design a Distributed Job Scheduler — Cron at scale, resilient task queues
- Design a Feature Flag / A/B Testing System — real-time flag delivery + experiments
- Design a Log Aggregation Service (Splunk-style) — write-heavy, indexed queries
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once
- Deep Dive: Consensus Algorithms — Raft (worked example), Paxos, leader election
- Deep Dive: Distributed Transactions — 2PC, sagas, event sourcing